

Webcam-based Road Condition Monitoring

Case Study Sensors and Actuators (SS22)

Final Report

- Anmol Sharma
- Elham Alfugara
- Tobias Dapolonia
- Wen-Hao Wu

Table of Contents

1. Abstract	3
2. Introduction	4
3. Datasets and Methodology	7
3.1. Data collection	7
3.2. Data Classification	8
3.3. Data annotation	9
3.4. Final Dataset	11
4. Model	12
4.1. Model Training	14
4.2. Final result of training	16
4.3. How the detection works	17
5. Set Up	18
6. The User Interface	20
6.1. User Interface screen and functions	21
7. Data preparation	24
8. Report Generation	26
8.1. Motivation	26
8.2. Main features	26
8.3. Methodologies	27
8.4. Problems and solutions	28
8.5. Conclusion	29
9. Result	29
10. Lessons Learnd	30
11. Future Work	31
11.1. Severity of road damages	31
11.2. Duplicate detection issue	31
11.3. Live mode with gps data	31
11.4. Mobile application conversion	32
11.5. Privacy issues	32
11.6. Weather condition detection	33
12. References	34
13. Appendix	35
13.1. Appendix 1 Timetable:	35
13.2. Appendix 2 Example Report:	36

1. Abstract

Road damage detection is a powerful tool to maintain the road quality which is a major issue for all the road users. Not only that, classifying the severity of the road damages is also an important topic for the authorities to recognize where or which part of the road should be prioritized to be repaired.

Inspired by recent success on applying convolutional neural networks on computer vision from other researchers, in this project, a road damage detection model has been created and trained, along with an user interface and an auto-generated report tool. The report starts with introduction to road maintenance issues and project management analysis, to the datasets that have been collected online and by ourselves, to the model that was chosen with the training results, to the set up such as camera selection analysis, to the code walk through that includes the user interface, report data generation, as well as the detection report generation tool. In the end, the report is summarized with results, conclusion, and the future outlook.

2. Introduction

Roads are one of the most valuable resources in modern civilization, boosting the economic benefits of numerous activity sectors, the quality of life of residents, and the growth of nations, with a particular influence in metropolitan areas.

However, pavement surface characteristics deteriorate due to exposure to abrasion from vehicle traffic, material aging, and harsh weather conditions. Cracks or other sorts of suffering (distortion, disintegration, etc.) may appear as a result.

These cracks in the pavement make people uncomfortable while driving, harm automobiles, and may cause more evasive maneuvers that could lead to a collision. Therefore, the Public Works Department or equivalent departments in every nation are particularly concerned with road maintenance. In order to preserve the road in the state it was originally built, safeguard nearby resources, guarantee user safety, and enable effective and convenient travel along the route, road maintenance is crucial.

For these departments, gathering information about pavement surfaces is a laborious procedure. They must spend money on human resources, specialized inspection vehicles, and numerous sensors or cameras. Besides, the results of the road inspection process are subjective due to different human views. Moreover, because people have diverse perspectives, the outcomes of road inspections are subjective.

In light of the aforementioned considerations, this project was chosen as the team case study. The chart below illustrates the project motivation.

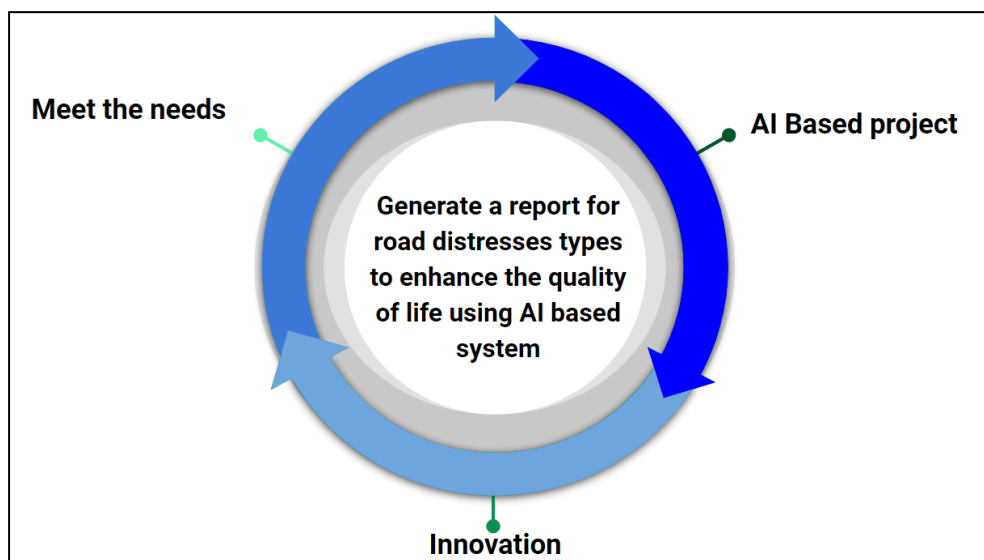


Figure : 1, Project motivation

Individual and team motivation is the primary element impacting the teams productivity and shaping the project scope.

However, this will lead to one of the most important project phrases, which is project scope where it outlines what is and is not included in the project in explicit terms and regulates what is added to or withdrawn as the project is carried out.

To execute this project successfully, the project scope was carefully thought out. And this is how it is defined:

Build A CNN-based module that classifies the asphalt pavement conditions into 7 categories with 3 classifications of severities using a smartphone camera and GPS data taking into consideration the weather factor as user input data.

The project scope could be divided into these main terms:

- Build a model to classify the distress for asphalt pavement into 7 categories.
- Classify the severity
- Weather consideration as user input.
- GPS data gathering

While the promised project deliverable is:

Generate a spatio-temporal report containing all the distresses classification found in a recorded video with their severity level.

The schematic and project summary are displayed in the image below.

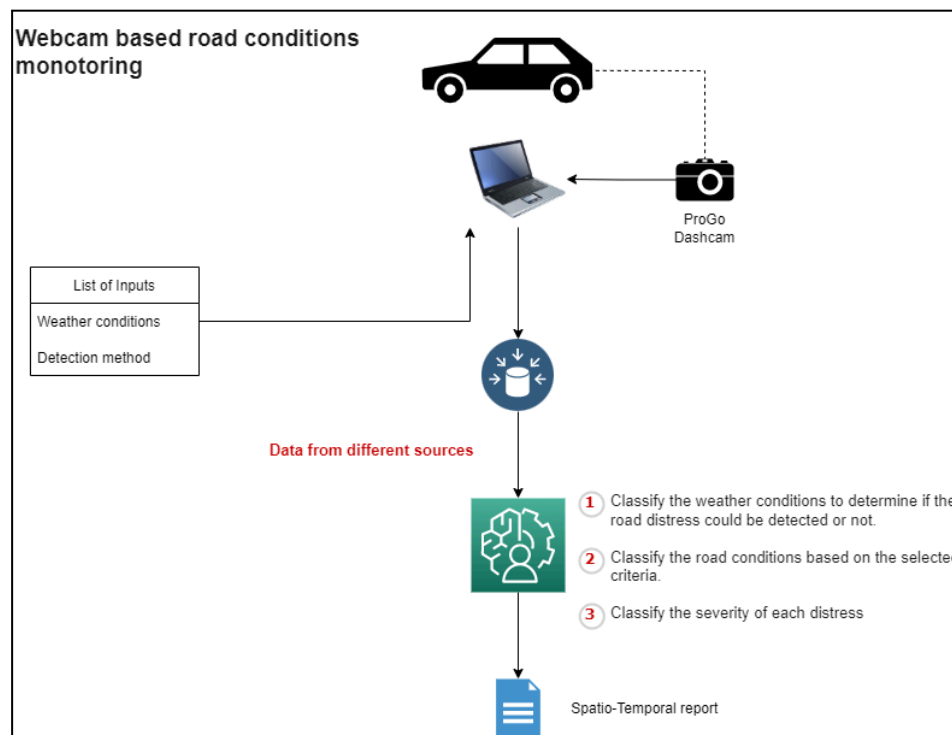


Figure : 2, Project schematic

To achieve the project goals and to follow the scope, the project was divided into iterations to make the work more flexible and easier to manage, furthermore, a timetable was set out the major milestones to track the progress of the project. The timetable can be found in the appendix.

Along with monitoring the progress of the project, risk treatment and risk management were given priority for each iteration. In addition, the ability to recognize the project's strengths, weaknesses, opportunities, and threats are made possible through the use of appropriate risk management procedures. The table below provides an overview of the main risks encountered throughout this project:

Risks	Risk Status	Risks treatment
Scope Creep	Good	Brainstorming and group meetings
Cost	Good	Funded by THD
Privacy and data protection	Medium	By re-location the camera, adjusting the camera angle, and not implementing a face-recognition algorithm
Under-represented classes	Medium	Image augmentation and manual labeling

Tabel : 1, Risks treatment

However, the project execution phases, including datasets collecting, model creation, user interface design, and final report generating, are detailed in this report.

3. Datasets and Methodology

This section shows how the data was collected and prepared. It also explains the various classifications into different categories and the problems faced during manual labeling.

3.1. Data collection

To train and test the model for road damage detection, image data was needed either from above the road surface or using on-board cameras on vehicles. Around 30,000 road damage images were prepared from 3 different dataset as listed below.

1. Road Damage Dataset 2020 - RDD-2020

The Road Damage Dataset 2020 (RDD-2020) is a large-scale heterogeneous dataset comprising 26620 images collected from multiple countries using smartphones. The images are collected from roads in India, Japan and the Czech Republic. This Dataset is labeled in 8 different categories.

2. German Asphalt Pavement Distress Dataset - GAPs

The German Asphalt Pavement Distress (GAPs) dataset covers 200 meters of asphalt roads with high resolution labeled images. The dataset is divided into two-parts: GAPs v1 and GAPs v2. GAPs v1 contain 1,969 gray valued images and GAPs v2 contain 2,468 gray valued images.

3. RSDD Image Dataset

This dataset has 6140 labeled images and it is classified in 4 different classes of road damages such as crocodile crack, lateral crack, longitudinal crack, pothole. Images are further divided into training set, validation set and testing set.

4. Dataset Expansion

In order to increase the dataset, a video of the road was made using a gopro camera installed on the front windshield of the car, converted the video into images, and then labeled each image manually according to the road damage classification.

3.2. Data Classification

Based on the “International Road Maintenance Handbook for Road Maintenance”, the road damage was classified in 4 different categories such as Linear crack, Alligator crack, Potholes, White lines. Each of these damages are described briefly below.

1. Linear Crack

A long way series of linear cracks running along the road surface. These cracks may appear as one or as a series of parallel cracks. Some tree roots, such as gum trees, cause single cracks, especially in areas with underlying soils such as clays that dry out. These cracks can also be due to poorly constructed joints in a road's surface.



Figure: 3, Linear Crack ¹

2. Alligator Crack

A crocodile crack is a series of interconnected cracks that looks like the skin pattern of a crocodile when it is formed. Typically, they form along the road's wheel path as a result of the damaging effects of traffic over time.

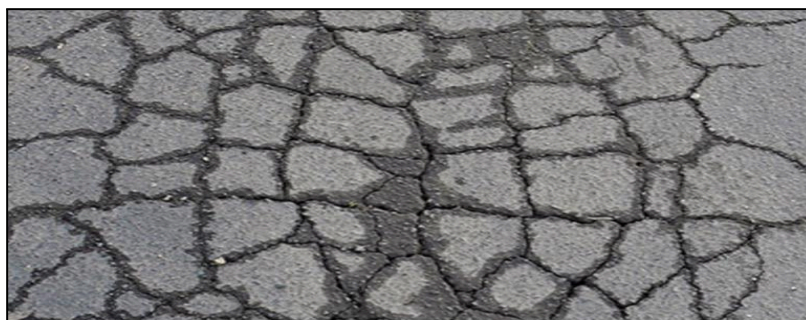


Figure : 4, Alligator Crack²

¹ [8]

² [8]

3. Potholes

A pothole is a bowl-shaped indent in the road surface, which is generally characterized by irregular or rough edges and nearly vertical sides at the top. Potholes are caused by water seeping through cracks in the road surface, and weakening the underlying road.



Figure : 5, Potholes³

4. Whiteline

The line on the road is called road marking. They provide information about the road and use it as a traffic sign. There are two kinds of road marking. One is permanent marking which is in white color and other is temporary marking which is yellow in color. In this project, broken whiteline was classified as one of the road damage as shown in figure 6.



Figure : 6, Whiteline ⁴

3.3. Data annotation

After comparing all the data and classes, there are some classes that were discovered under-represented. In order to balance the different classes, the road images were manually

³ [8]

⁴ [8]

annotated on a platform called PLAINSIGHT. The annotation pipeline is presented in Fig.7. Annotation is a labor-intensive procedure and took considerable time because the complete dataset was first manually reviewed on the screen fig 7 (a) and then all recognized deficiencies were annotated by enclosing them with bounding boxes, fig 7 (b) and finally classified by attaching the proper class label, fig 7 (c).

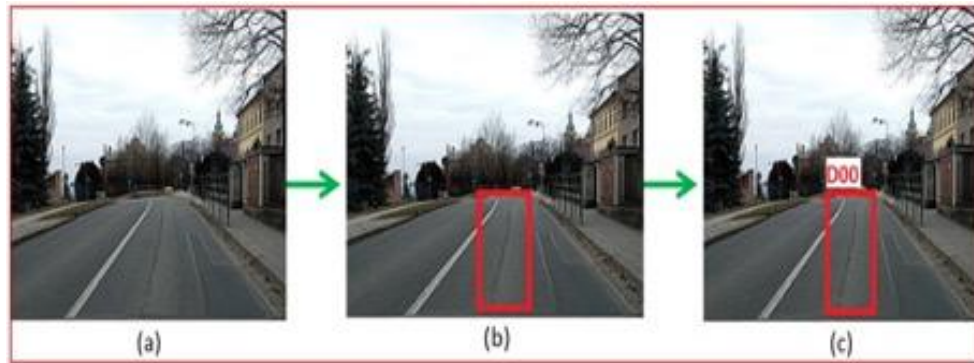


Figure : 7, Manual Labeling

Images are classed into four different classes with a class name as shown in Table.2

Clas	Description	Class name
0	linear Crack	#0ea91f
1	Alligator crack	#d81212
2	Patch	#f5a623
3	Potholes	#bd10e0
4	White line	#50e3c2

Table: 2, Different class names

3.4. Final Dataset

The final dataset is a combination of the datasets mentioned in 3.1. It contains 11.947 pictures with 26.191 annotations. For the training this number was further increased with augmentations to over 30.000 pictures and over 100.000 annotations. For this the online annotation tool Roboflow was used.⁵

The class distribution can be seen in Figure 8. At the first impression it looks like the pothole class is over-represented, but this was done on purpose, to compensate for previous under-representation. This can be explained by taking a look at the augmentation steps.

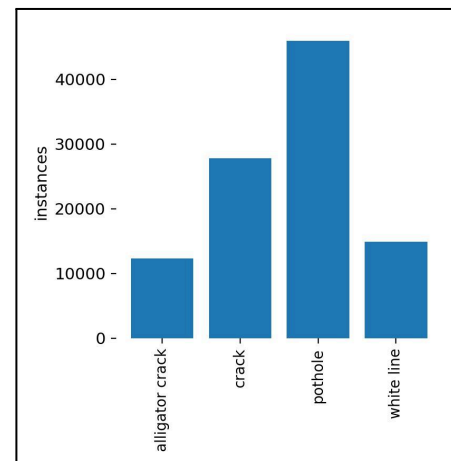


Figure: 8, class distribution

While mosaic and cropping augmentation steps were used for the whole dataset, to reach from 10k to 30k pictures, color changing, resizing, scaling, flipping, sheareding, translation, and combinations of the above were used to augment the pothole pictures. The possible augmentations can be seen in figure 9. But since the augmented pictures are less valuable to the training the results for the pothole recognition in comparison to the other classes in the confusion matrix was worse. Therefore the decision was made to further increase the number of augmented pothole pictures, until the recognition results used to be more equal.

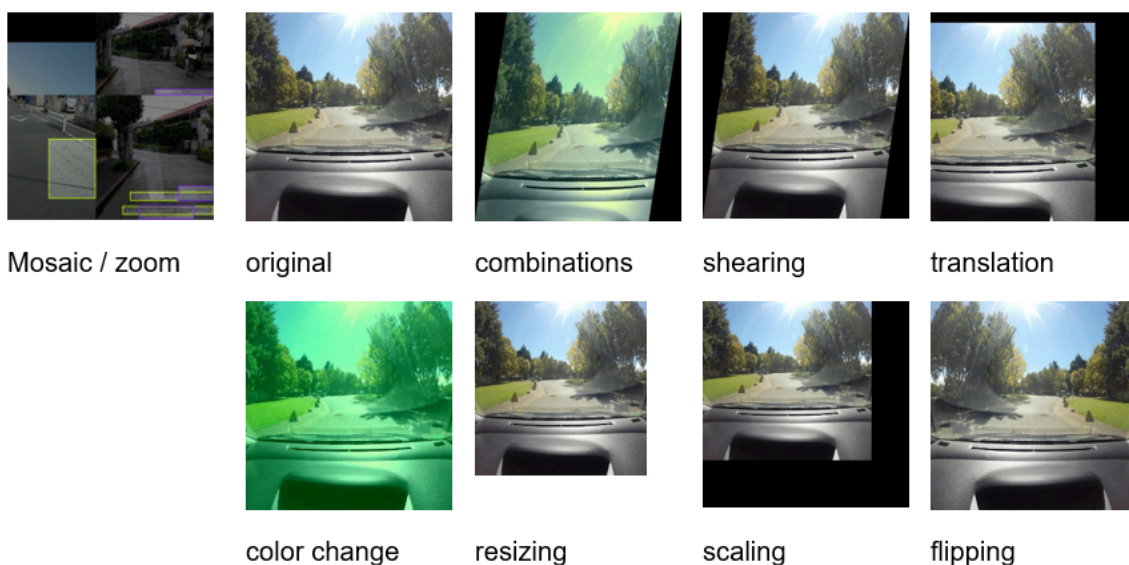


Figure: 9, Augmentation

⁵ [5]

4. Model

For the Image recognition YOLOv5 (You only look once) was used. YOLOv5 is open source and a good start point for fast and good results. The framework allows you to choose between 5 different models.

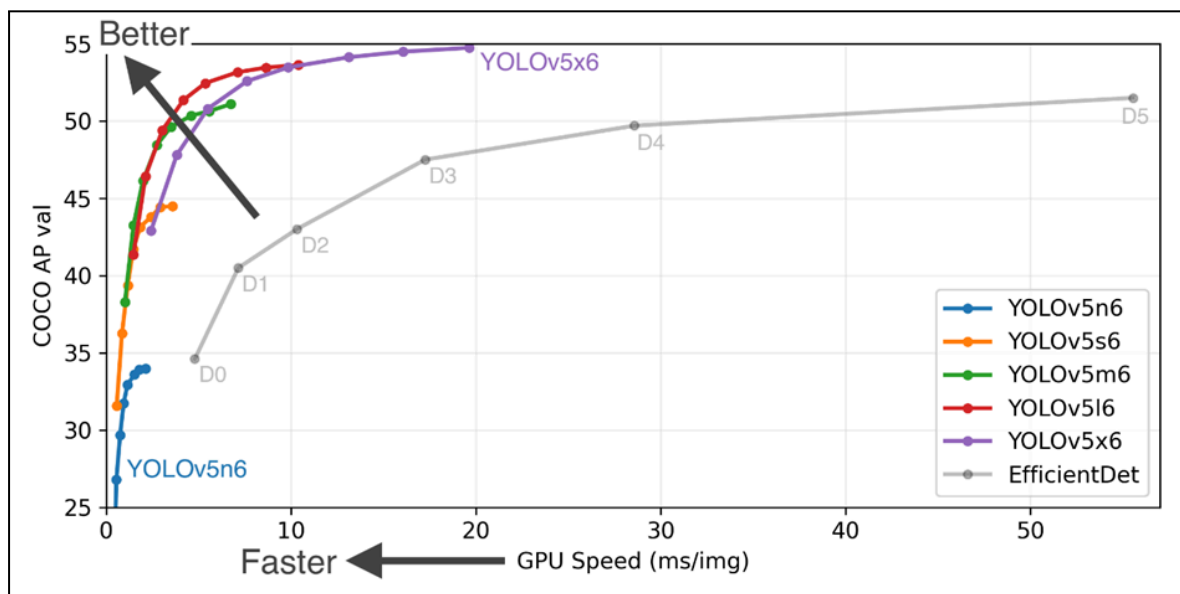


Figure: 10, YOLOv5 Model Overview⁶

From YOLOv5n6 to YOLOv5x6 the models increase in complexity and theoretical recognition power, but are also more sensitive to overfitting and need more calculational power for training and executing. After trying different types of models with different datasets, the best results were achieved by using the YOLOv5s. With the YOLOv5m the training duration was too long and the prediction very slow. Therefore to create the report in a reasonable amount of time the decision was made to implement the YOLOv5s. With more calculation power the YOLOv5m version could be implemented, which could help to further increase the recognition.

⁶ [7]

4.1. Model Training

To reach the final dataset mentions in 3.4 multiple different steps were made to face the following issues:

1. The creators of the datasets used to label different classes which were missing in other datasets or they used to be named differently.
2. The model used to confuse longitudinal and lateral cracks therefore it was decided to combine those to one class, since those are basically identical and just rotated.
3. The distribution of the classes used to be unbalanced. For example, the RDD dataset used to have ~467 occurrences of lateral cracks but 7339 occurrences of crocodile cracks. Similar issues used to be with potholes and white line damages in other datasets.

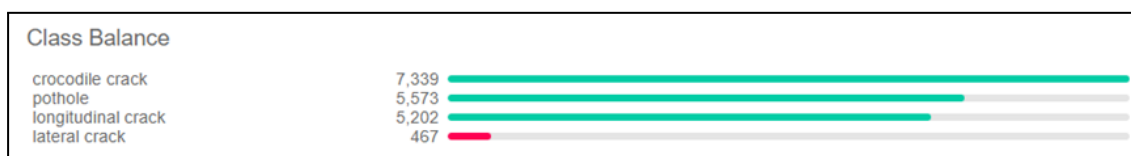


Figure: 11, Class Balance

4. The quality of the validation subdataset was not sufficient. If a class is under-represented in the validation data or the pictures are too different from the actual use case the training results, represented in the confusion matrix, will give a wrong picture of the actual reality.

In addition to adjusting the dataset, the following training parameters used to be adjusted to improve the results:

1. Number of epochs:

Different numbers of epochs were tried to increase our predictions for the classes. But with the increasing size of the datasets the time needed per epoch increased significantly. So with the final Dataset of roughly 30.000 pictures and trained for 50 epochs.

2. Frozen layers:

To improve the result and still stay in a training time of about one day. A basic training of 50 epochs was made with the 30k dataset and used frozen layer learning to further train the model with a smaller augmented dataset. This further improved the results and decreased the time of training significantly.

3. Batch size:

The batch size was increased to 16 since that was the maximum value the RAM was able to process.

4. Image size:

The standard for the YOLO model is 640x640. Experiment to increase that was made but this led to no improvement and decreased the speed significantly. In addition, a lot of the pictures in our dataset used to be in the size of 640x640.

Together with the final application a selection of folders containing the results of different training runs can be found in the application folder. This selection was limited, due to the size of the folders.

4.2. Final result of training

The results of the training can be shown best in the confusion matrix. In all of the classes, a recognition rate of roughly 80% was achieved. The only issue with our final results is that a high percentage of noise is classified as damage. But these results are also difficult to judge since some of the pictures in the dataset are poorly labeled. To have more significant results the validation dataset needs to be adjusted and manually filtered.



Figure: 12, Confusion matrix

4.3. How the detection works

The Yolo model is started with the run_yolo function, which is triggered by the user interface. This function guarantees that the right flags are set for the YOLO detection. Everything in curly braces are variables which are set in the config.py file.

```
python "{detect_file_path}" --weights "{weights_file_path}" --img {imgagesize} --conf {confidence} --source "{source_file_path}" --save-txt --save-crop --project RDD --view-img
```

Figure: 13, command to start YOLO detection

Explanation of the flags

--weights: With this flag the yolo detection can be run with different weights. Those can be configured in the cofig.py file. Usually, the files containing the weights should be saved in the YOLO/weights folder.

--img: This parameter can be adjusted in the config.py file and contains the information, with what resolution the detection should be executed. For this parameter it is recommended to be equal to the resolution during the model training.

--source: This parameter contains the path to the video file, the user selected in the interface

--conf: This parameter can be adjusted in the config.py file and contains the confidence level for detection

--save-txt: With this flag the YOLO model is saving the bounding box information in a .txt file. This is necessary for the following analysis of what was detected and which picture contains the most information.

--save-crop: If this flag is saved a cropped version of the detection can be saved. These crops could be used in the future for further analysis (severity or confirmation). Until now this flag is used to save the full scaled pictures for the report.

--project RDD: With this flag the information is always saved in a central and well named folder, which is important for the further analysis of the data.

--view-img: This flag is used, that the user can see the detection process of the yolo detection.

In the future those adjustments of the config.py file could be placed in a setting screen in the user interface.

5. Set Up

To increase the data collection and to check how the model will detect the road damages live, a camera-based approach is adopted. Different types of cameras were analyzed and compared with following factors: Resolution, Location accuracy, Cost and Implementation Challenges as shown in the table below.

Camera Options	Resolution	Location accuracy	Cost	Implementation challenges
GoPro	High	High (At low speed)	High	
DashCam	Good	High (At low speed)	Affordable	
Smartphone	High	High	Low	Only for offline mode

Tabel: 3, Camera comparsion

After analyzing all the different camera options, it was concluded to go with Go pro model 9 due to its high resolution, high location accuracy and less vibration and shaking effect.



Figure: 12, Go Pro Camera

Go pro 9 also provides continuous GPS information which is needed to generate the report for our model.

The camera was mounted on the front windshield of the car as shown in fig13. This installation captured all of the road damages, the only problem with this installation was that our model also detected the reflection due to wind screen as a road damage.



Figure: 13, Camera Setup

6. The User Interface

User interface is important to meet end-user expectations and support the practical functionality of our project. A well-executed user interface facilitates effective interaction between the user and the AI model and the final report through contrasting visuals, clean design, and responsiveness.

The user interface was created to fulfill and meet the project scope which states that the weather condition should be provided by the end-user, in addition to predicting the asphalt distresses in both live and recorded video modes.

Creating the user interface started by determining the required inputs for the YOLO model and the final report-generating function.

For the YOLO model:

1. Detection mode.
 - a. Live mode
 - b. Recorded video mode
2. Video path/location, needed in case of recorded video mode.

For final report generating:

1. Company name
2. Detection mode
3. Weather condition

The above-mentioned parameters are used to run the (run_yolo_and_generate_report) function.

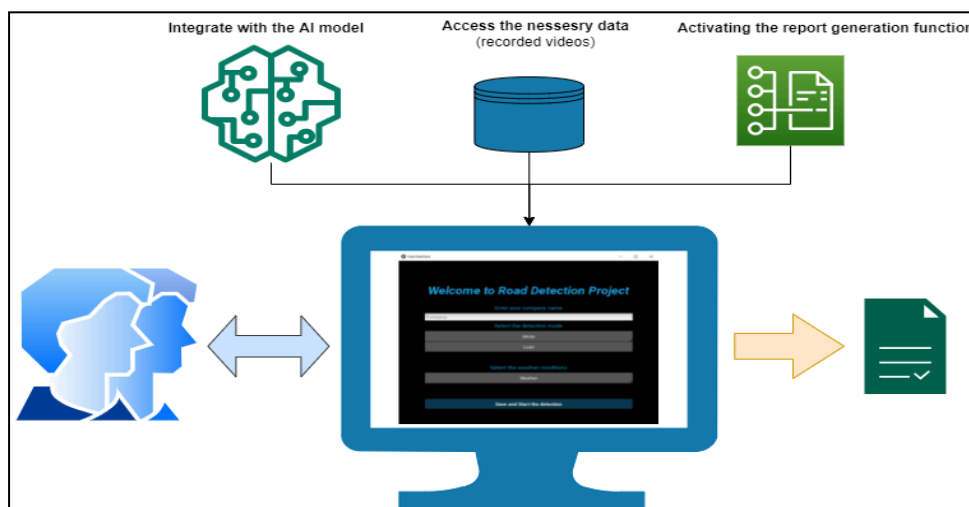


Figure: 14, User Interface diagram

6.1. User Interface screen and functions

In this section, the interface screen content and functions will be explained.

Starting with the main screen, this screen contains the following:

#	Widget type	Widget title	Purpose	Values
1	Label	self.greeting	Greeting statement	
2	Label	self.Info	Guide statement	
3	TextInput	self.user	To insert the (Company name)	
4	Label	self.Info2	Guide statement	
5	Spinner	self.spinner2	Select the detection mode	Recorded video Live mode
6	Button	self.button3	Open the file chooser pop-up	
7	Label	self.Info3	Guide statement	
8	Spinner	self.spinner	Select the weather condition	Good Bad
9	Button	self.button	Start the prediction and generating report	

Table: 4, Elements of User Interface

The picture below shows the main screen layout and widgets.

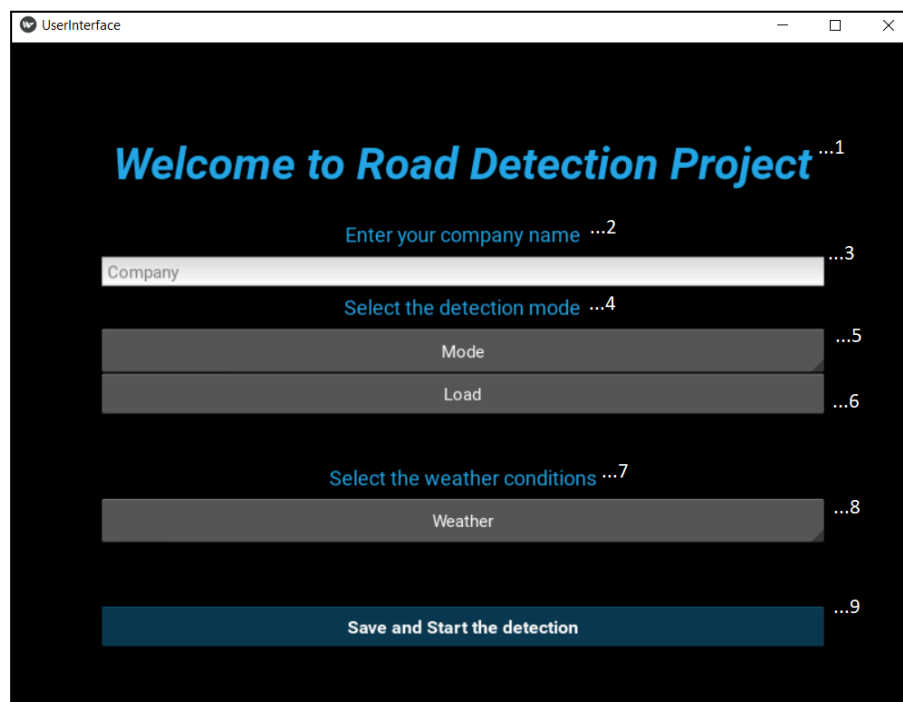


Figure: 15, main screen layout

The steps for starting prediction:

1. Fill in the company name
2. Select the detection mode
3. Choose the video for recorded video mode
4. Select the weather condition
5. Start the detection

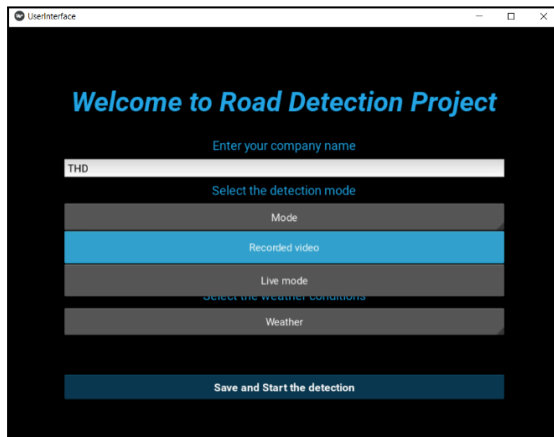


Figure: 16, Mode selection

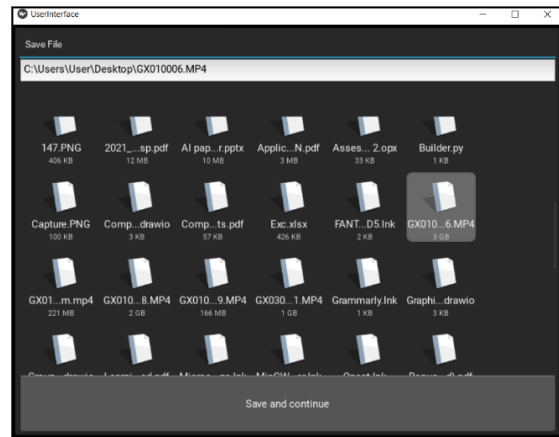


Figure: 17, Path selection

Objective and accurate detection or evaluation of road distresses is an important task in pavement maintenance and management. This detection process is directly influenced by the weather condition as bad weather conditions affect the visibility of pavement surface features. For this reason, the risk of misclassification due to the bad weather was eliminated by asking the end-user to select the weather condition before starting the prediction process. As long as the weather is in good condition the prediction will start, otherwise, it will not. The pictures below illustrate the user-Interface output message for bad and good weather conditions:

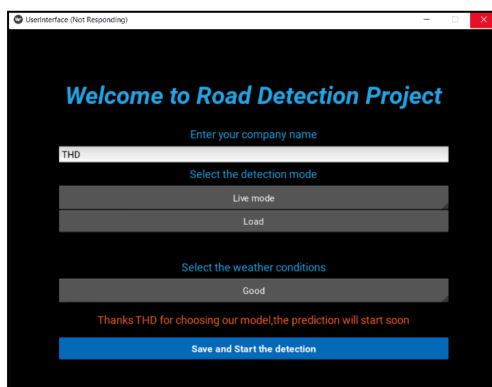


Figure: 18, Weather condition good

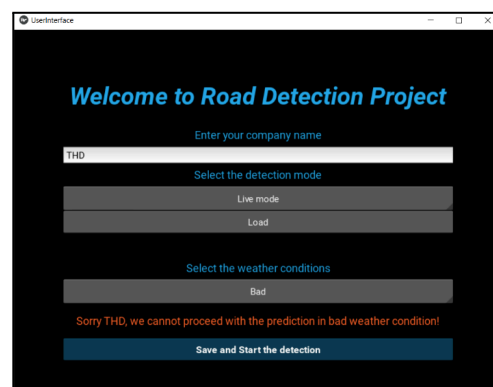


Figure: 19, Weather condition bad

After starting the detection, the immediate results will be shown on a pop-up screen:

1. For recorded video mode, the video will start, and the detected distress will be bonded by boundary boxes.
2. For live mode, the pre-enabled camera will work, and the detected distress will be bonded by boundary boxes.

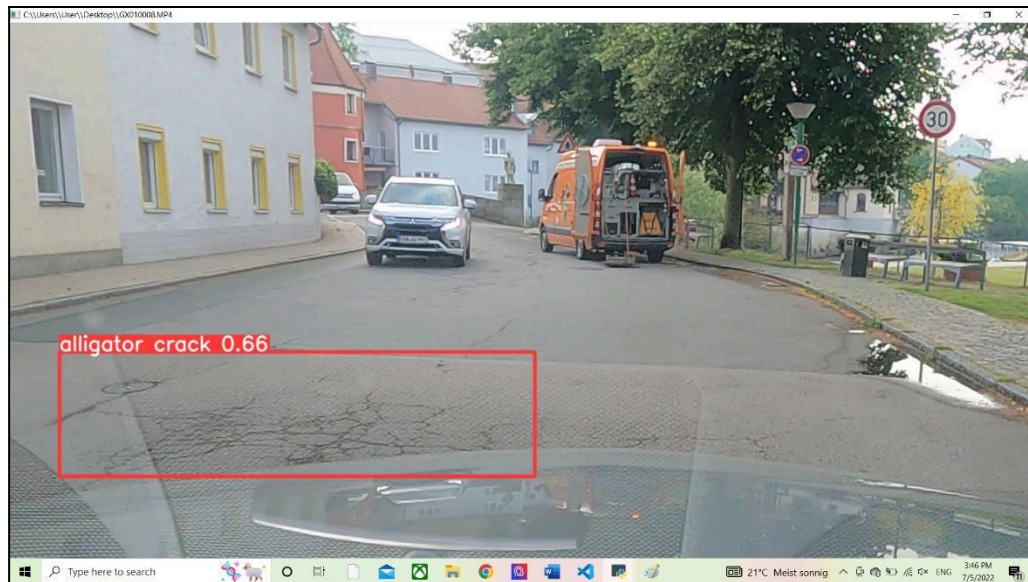


Figure: 20, Pop up of detection

To stop the prediction for both recorded video and live modes, press (q) which stands for (quit), thereafter, the report will be generated.

```

139 #select folder button
140 self.button3=button(text='Load',size_hint=(1, 0.4),disabled =cond1)
141 self.button3.bind(on_press=self.callback2)
142 self.window.add_widget(self.button3)
143
144
145
146 # Show the file link
147 self.info2 = Label(text="",
148                    font_size=18, color="#25a9e8", size_hint=(1, 0.4))
149 self.window.add_widget(self.info2)
150
151 # Select the weather condition
152 self.info2 = Label(text="Select the weather conditions",
153                    font_size=18, color="#25a9e8", size_hint=(1, 0.4))
154 self.window.add_widget(self.info2)
155
156 # Select the weather conditionq
157 self.spinner = Spinner(
158     # default value shown
159     text="weather",
160     # available values
161     values=["good", "bad"],
162     # just for positioning in our example
163     size_hint=(0.1, None),
164     size=(100, 40),
165     pos_hint={"center_x": 0.5, "center_y": 0.5})
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Figure: 21, Feedback in Command prompt

7. Data preparation

To create the report the data needs to be prepared and cleaned. In this process, several things are necessary to do. To make the code easy to understand, those steps are separated and done in separate functions:

```
clean_report_data_folder()

create_metadata_file(companyName, DetectionMode, WeatherCondition)

create_gpx(source_path)

create_segment_Folders(NrofFrames, SegmentSize)

create_frontpage_and_segmet_data(source_path, NrofFrames, SegmentSize)

create_report(source_path)
```

Figure: 22, Functions for data creation

`clean_report_data_folder`: This function deletes the report data folder and recreates it with just the basic subfolders. This is done because depending on the settings, the segment size and meta data can be different. To guarantee that the report just contains the information for the current detection, the cleaning of the old data is necessary.

`create_metadata_file`: This function creates the .txt files with the meta information (company name, detection mode, weather condition) which are later used for creating the report.

`create_gpx`: This function creates a .gpx file based on the metadata of the video. Since the gopro is saving continuous gps information the program can create a complete file, with which the program can comprehend the route driven by the user and the gps locations of the detected damages. To extract this information the “exiftool” was used.

`create_segment_Folders`: This function is creating the necessary folder structure for the report. Depending on the video length and the number of segments the folders Segment1-Segment(n) are created.

`create_frontpage_and_segmet_data`: This function is used to fill the folders, created in the previous step, with data. In the first step the information for the front page is created. This contains the damages detected and their number of occurrences and the picture with the road and locations. The damage frequency and class are counted in the .txt files from the YOLO detection. To create the frontpage picture, “folium map” was used. This library can be used to interact with maps and visualize data in it. After some preparation this data is saved in an .html file. With the “gecodriver” library this html file is automatically opened in mozilla firefox and a screenshot is created and saved.

To place the pictures in the right segment folder a loop is started. This loop gets from 0 to the number of frames. if the loop variable matches the frame number, in which a damage was detected, the following steps are initialized:

1. The number of damages for the frame are counted
2. If there is already a picture in the segment folder the number of damages recognized in that picture is compared with the number of the previous.
3. If the number of damages in the picture is higher the picture is replaced. like this the report will always show the picture with the highest amount of information.
4. The gps location of the picture is associated and the corresponding gps location and the corresponding address is written in the location.txt file.
5. The info.txt file is written.

After those preparation steps the creation of the report will be initialized.

8. Report Generation

In this project, a tool to automatically generate a report after detection was created. In the following chapter, the motivation to create this tool, the main features of the tool, the methods that were used to create this tool, and the problems faced while creating the report will be elaborated.

8.1. Motivation

First of all, the motivation of creating an auto-generated report after the detection is based on the recording/documenting purposes. The end-user of our product was assumed as the traffic department of the government. Therefore, a summary of the detected distress at a specific location will be relevant for road maintenance and repairment.

8.2. Main features

Second, the main features of this tool are as follows: Display of the end-user's association's name, display of the detection mode, display of weather condition, a summary of the total amount of distresses detected during the detection(in category), a visual display of the route of the drive, display of the recorded date and time, visual displays of the detected distresses, and the display of the exact location where the distress was detected(Figure 23, Figure 24).

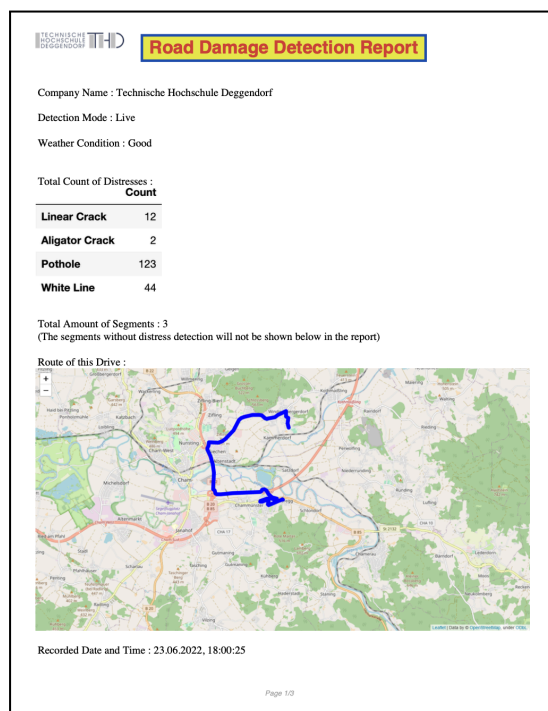


Figure :23, Cover page of detection report

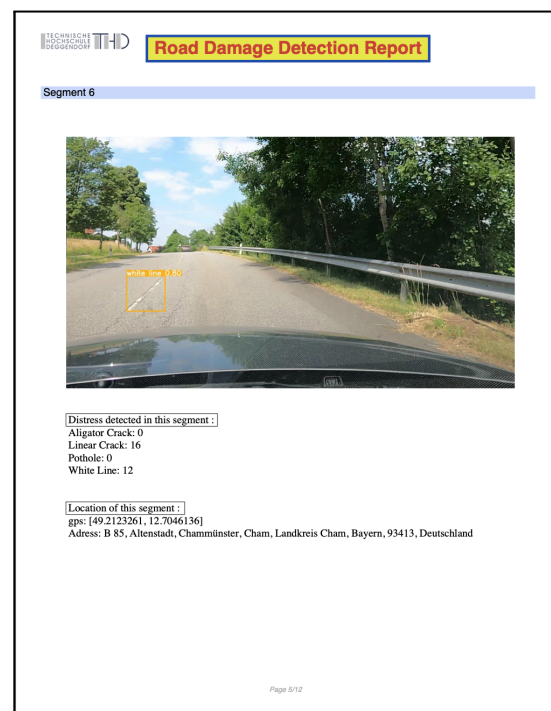


Figure: 24, Detection instance with location details

8.3. Methodologies

Third, the detection report application was made based on Python and its libraries (Table 5). The main libraries used were: fpdf2, datetime, pandas, dataframe_image. The fpdf2 library is mainly used to create the pages of the pdf as well as the structures of the pdf and how the pdf should look like. The datetime library is used to extract the current date and time as the report was created and will afterwards be printed out. Pandas and dataframe_image are the libraries that were used to deal with the counting of the distresses. In the cover page of the report, there is a summary of all the counted distresses, and that was done by these two libraries. Pandas was used to create a dataframe table and the dataframe_image library was used to save the table as an image. Afterwards, the saved photo of the summary will be printed out too.

Library used	Functions
Fpdf2	To create pdf pages and it's structure
Datetime	To extract date and time information
Pandas	To create a table for the counted distresses
Dataframe_image	To create and save an image of the counted distresses

Table: 5, Libraries used and their functions

Furthermore, some important functions that were defined and will be explained more in detail (Figure 25). The header function controls the font size, font color, border style of the title, as well as the THD logo on the top left corner of the report. The footer function creates the page count for each page as well as its font color and font size. The chapter_title function creates the secondary title of each page after the cover page as well as its font color and font size. The date_time function extracts the current date and time and stores it in the variable as well as subsequently printing it on the pdf. The chapter_body function reads two text files which are the location files and the distress counting files and prints them in a specific font style. Additionally, it loads a distress image and prints it on the pdf page. Last but not least, the print_chapter function wraps up the chapter_title function as well as the chapter_body function so that they could be called at once.

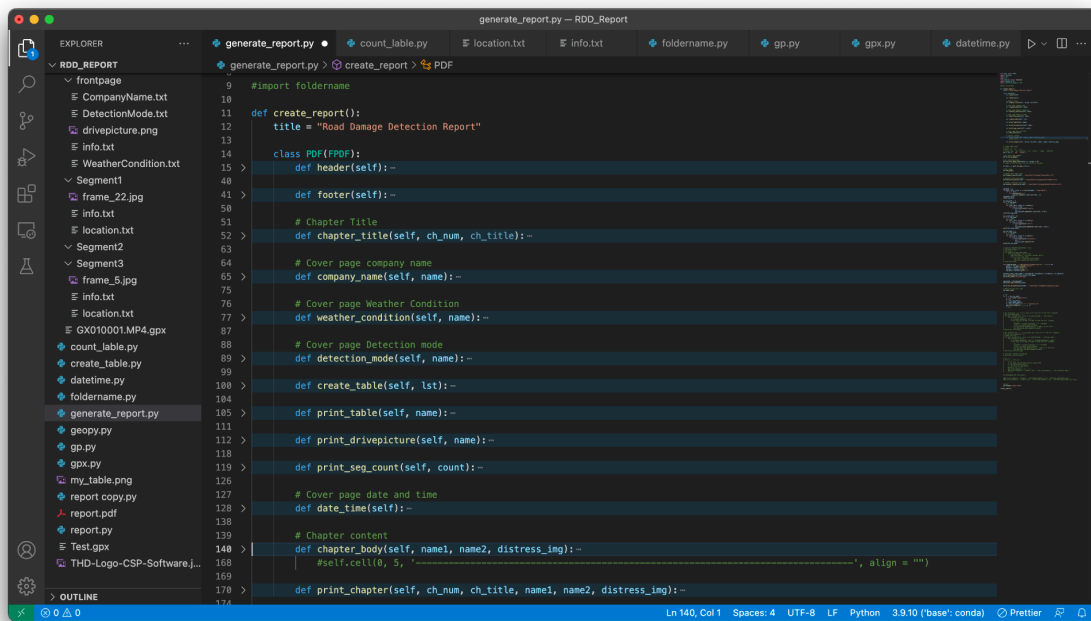


Figure: 25, Overview of all the defined functions

8.4. Problems and solutions

In addition, many problems were faced during the project, such as the duplicated detection issue. The duplicated detection issue was caused by the fact that the detection was made every frame. Due to the fact that the same distress could appear multiple times in consecutive frames, the same distress could be detected multiple times. A semi-adequate solution from the report's side is to create so-called segments. Segments are created based on a certain amount of frames, and the detection video will be split into snippets according to that. The segments should be created in a manner that it should not be too large, otherwise the gps location wouldn't be accurate anymore (The gps location is extracted at the mid-point of the snippet). Additionally, count of distresses for each segment are also displayed. This sort of solved the multi-detection issue since the end-user could just look at the frequency or the total count of the distresses detected in that segment to figure out which part of the drive was mostly damaged.

Furthermore, some algorithms were created to loop through folders and files inside the folders. The code eventually grew huge due to stacking "for" loops on top of each other, and became almost impossible to interpret by an inexperienced programming student. Naming convention was also an issue when lines of codes stack up and exchanging codes between project mates. This is when the debugging tool comes into rescue. The debugging tool allows the code to run one line after another, therefore, to find out where the mistake was, became possible.

8.5. Conclusion

In conclusion, the auto-generating report application was made for the end user to realize what kind of damages are there, being able to locate where the damages are, and being able to visualize where the detection was made and how the distresses look like.

9. Result

After the code was finished there were some tests conducted in the region of Cham. What can be said is that, the code works and produces the required pdf file describing the road situation. One of the examples can be seen in the attachments. However, during the test, there were some downsides as well. In general the code is not very robust and crashes in some situations. All problems of the code couldn't be removed or even found, due to the limited time available. To make the code more robust and user friendly a bigger debugging and testing phase would be required. The limitations found until now are as follows:

1. Until now the code only works with go pro videos, because no gpx file can be created with other videos.
2. The time for analyzing the video is quite long.
3. No gps information is conducted during the live mode, therefore the code crashes as well.

In conclusion, that means that the current code works well as a prototype or under supervision of people that know the limitations. The next step would be to make a detailed test series and fix the known problems to make the product accessible for everyone and user friendly.

10. Lessons Learnd

The knowledge learned from carrying out a project is known as lessons learned. Both the advantages and disadvantages are included. The goal is to build on what worked well in the past and avoid making the same mistakes. The remarks and suggestions that can be learned should be identified.

- Computational and Logical thinking from coding

Computational thinking comes before programming. It involves breaking down a complicated problem into manageable components that a computer could understand. This way of thinking was gained through this project.

- Team working and building

The capacity to recognize and inspire each team member during this project to create a cohesive, productive, and successful team.

- Clear communications

One of the most crucial lessons acquired via this project is to keep lines of communication open. This will impact the outcomes and the spirit of the team

11. Future Work

During the project, there were many issues and new ideas every now and then. In this chapter, the summary of these issues and ideas will be elaborated and discussed.

11.1. Severity of road damages

First of all, the project failed to define the severity of the road damages. This comes with two reasons. First, how is severity defined? Does it depend on how often a road damage occurs? Or does the severity base on the size of the road damage? If it depends on the size of the road damage, the size of the road damage could change based on the camera angle and the distress becomes bigger as the vehicle approaches. All in all, this is a challenging topic to solve.

11.2. Duplicate detection issue

Second, as mentioned before, the duplicated detection issue. As the detection was made every frame of the video, and due to the fact that the same distress could appear in consecutive frames, the same distress could be detected as a new distress again. The solution to this could be to implement an algorithm that could memorize the specific road damage that was already detected. Therefore, the same distress wouldn't be detected as new distress again. However, this would be rather challenging due to the fact that a road damage does not have so many features for the algorithm to memorize. In contrast to human-recognition with memory, humans have different colors of clothing, different hair styles, different shapes of faces, and so on. In Addition, the pattern for the road damage could also slightly change as the vehicle approaches. This could also cause some issues. Another approach could be simply reducing the detect rate per frame, with the cost of high possibility to miss some actual new detections.

11.3. Live mode with gps data

Third, the application at the moment wouldn't work in live mode. Due to the fact that the gps data couldn't be extracted during a live detection. Without the gps location, the detection report wouldn't be generated correctly. The approach to solve this is to find a way to run the detection and simultaneously extract gps data to therefore create the detection report. At the moment, how the codes were set up was, as the end-user ends the detection, the system deletes all the previously saved data and writes in new ones. If the gps data is being recorded during the detection, they will be deleted as long as the detection is finished. There

should be a way to store the gps data in order to not get deleted or to only save the gps data after the previously saved data.

11.4. Mobile application conversion

Fourth, to convert this project to a mobile application. At the moment, this application would only run on computers. It wouldn't even run on some of the macOS machines due to the lack of library support. It would be a very practical thing to be able to transfer the whole system to a mobile operating system. So that the users could just place their mobile phones on their car and start the detection. It would however be very challenging but a very practical thing to do.

11.5. Privacy issues

Fifth, privacy issues. As this project was made in Germany, where people value personal data more than most things. People's houses, faces, car plates, etc. could be recorded during a road damage detection process. The idea to solve this issue could be to implement face detection, car plate detection, and so on. However, these algorithms are not going to be implemented to further increase privacy issues, but to reduce by detecting and blurring the private objects (Figure 26, Figure 27).

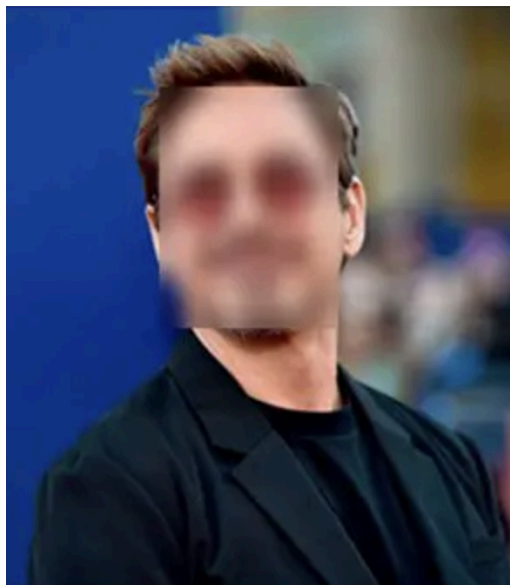


Figure:26, Blurred face of pedestrians, captured⁷



Figure: 27, Blurred car plate, captured⁸

⁷ [9]

⁸ [10]

11.6. Weather condition detection

Last but not least, the end-user needs to select the weather condition in the user interface before starting the detection. For now, if the user selected the bad weather condition, the program wouldn't start. It would be nice to also implement some sort of weather detection algorithm, so that the user wouldn't need to select a weather condition before the detection and system could detect that on its own.

12. References

1. Arya, D., Maeda, H., Ghosh, S. K., Toshniwal, D., Mraz, A., Kashiya, T., & Sekimoto, Y. (2021). Deep learning-based road damage detection and classification for multiple countries. *Automation in Construction*, 132, 103935.
doi:10.1016/j.autcon.2021.103935
2. Zhang, Lei; Yang, Fan; Daniel Zhang, Yimin; Zhu, Ying Julie (2016). [IEEE 2016 IEEE International Conference on Image Processing (ICIP) - Phoenix, AZ, USA (2016.9.25-2016.9.28)] 2016 IEEE International Conference on Image Processing (ICIP) - Road crack detection using deep convolutional neural network. , (), 3708–3712. doi:10.1109/ICIP.2016.7533052
3. Maeda, H., Sekimoto, Y., Seto, T., Kashiya, T., & Omata, H. (2018). Road damage detection using deep neural networks with images captured through a smartphone. *arXiv preprint arXiv:1801.09454*.
4. Zhang, C. (2021). Application of Convolutional Neural Network (CNN) Models for Automated Monitoring of Road Pavement and Winter Surface Conditions Using Visual-Spectrum and Thermal Video Cameras.
5. 2022 Roboflow Inc., <https://roboflow.com/> , accessed 07.07.2022
6. Ce Zhang, Ehsan Nateghinia Luis F. Miranda-Moreno, Lijun Sun. Pavement distress detection using convolutional neural network (CNN): A case study in Montreal, Canada
7. <https://github.com/ultralytics/yolov5>, accessed 07.07.2022
8. <https://caportal.com.au/vicroads/maintenance/gallery>, accessed 07.07.2022
9. <https://pyimagesearch.com/2020/04/06/blur-and-anonymize-faces-with-opencv-and-python/>, accessed 07.07.2022
10. <https://platerecognizer.com/blur-license-plate>, accessed 07.07.2022

13. Appendix

13.1. Appendix 1 Timetable:

13.2. Appendix 2 Example Report: